RaktSetu: Project Report

Expiry-aware blood redistribution network

Submitted by:

Pritika Khandelwal (R037)

MBA Tech AI B1, MPSTME

Abstract

RaktSetu finds blood units that are about to expire unused and moves them, with staff approval, to the centres and hospitals that can use them. Blood centres in India discard large volumes of expired components, and platelets last only 5 days. Existing systems such as e-Raktkosh show where stock is, but not which units will be wasted or where they should go.

The project is built in Java 21 as seven modules (a domain model, a decision engine, file persistence, a digital-twin simulator, a 16-option console application, a Spring Boot web server and a benchmark suite) plus a React front end. It supports three roles (network administrator, blood-centre staff, hospital staff), records every stock movement on a SHA-256 hash-chained audit log, and sends urgent SMS or WhatsApp alerts.

On a simulated Pune district network, expiry-first redistribution cut expired units by about 60% and halved unmet demand over 120 days. Profiling-led tuning made the engine 5 to 66 times faster at 1,000 blood centres, and the server handles 4.4 times more requests than before tuning, with no errors at 200 simultaneous users. The project is tested by 189 automated tests, with at least 80% line coverage enforced on every module.

1. Introduction

Blood is wasted mostly because it sits in the wrong place: one centre holds units it cannot use before they expire while another, a few hours away, runs short.

1.1 Problem

- Each component has a short, fixed shelf life: platelets 5 days, whole blood 35, red cells 42, plasma a year.
- Supply follows donation camps and routine dispatch, not where demand is. Rural centres often receive more than their hospitals use.
- Stock reports from centres are often days out of date, so a planner cannot trust what it sees.
- Blood collection in India rose from 126.95 lakh units in 2023 to 157.14 lakh in 2025, while discards rose from 8 lakh to 17.43 lakh; expiry was the most common reason, at about 46% of discards [8].

1.2 Gap

e-Raktkosh, India's national blood bank system, shows how much stock each blood bank holds. It does not predict which units will expire unused, and it does not recommend where they should go.

1.3 Objectives

1. Model blood components, facilities, stock and compatibility with object-oriented design.
2. Identify units at risk of expiring unused from stock levels and recent usage.
3. Allocate emergency and routine requests from the nearest compatible stock, soonest-expiring first.
4. Plan transfers of at-risk units for staff approval, comparing more than one strategy.
5. Keep a tamper-evident record of every movement.
6. Deliver it as a working multi-user web application, and measure its effect in a digital-twin simulation.

2. Requirements

Three kinds of user share one network, and each sees only what their job needs.

2.1 Users and roles

Role	Works at	Can do
Network administrator	The whole network	Set up the network; add facilities and people; plan, approve and reject transfers; see every request; read the audit log and alert status
Blood-centre staff	One hub or storage centre	Log collections (with the bag's donation number); report daily stock; dispatch requests; receive transfers; discard damaged units

Role	Works at	Can do
Hospital staff	One hospital	Raise emergency or routine requests; cancel before dispatch; confirm receipt; check network availability and compatibility

2.2 Functional requirements

1. **Stock:** record units by component, blood group and collection date; work out expiry from shelf life; remove expired units daily as wastage.
2. **Compatibility:** apply red-cell and plasma compatibility rules (plasma reverses them); issue O- to non-O- patients only as a last resort.
3. **Allocation:** serve a request from centres within 90 minutes, nearest first for emergencies and exact group first for routine requests, soonest-expiring units first; all or nothing.
4. **Expiry risk:** flag units unlikely to be used before they expire, from stock and forecast demand.
5. **Transfer planning:** propose transfers of at-risk units to centres expected to use them (Nearest-First and Expiry-First strategies); staff approve each one.
6. **Data freshness:** treat a centre whose stock report is over 3 days old as unverified: search it last and send it no transfers.
7. **Audit:** record every collection, issue, transfer and discard on a hash-chained log that detects edits.
8. **Accounts:** first-run setup of an administrator; one-time passwords; forced password change; account lockout after repeated failures.
9. **Notifications and alerts:** in-app notifications for each workflow step; SMS or WhatsApp alerts for urgent ones.

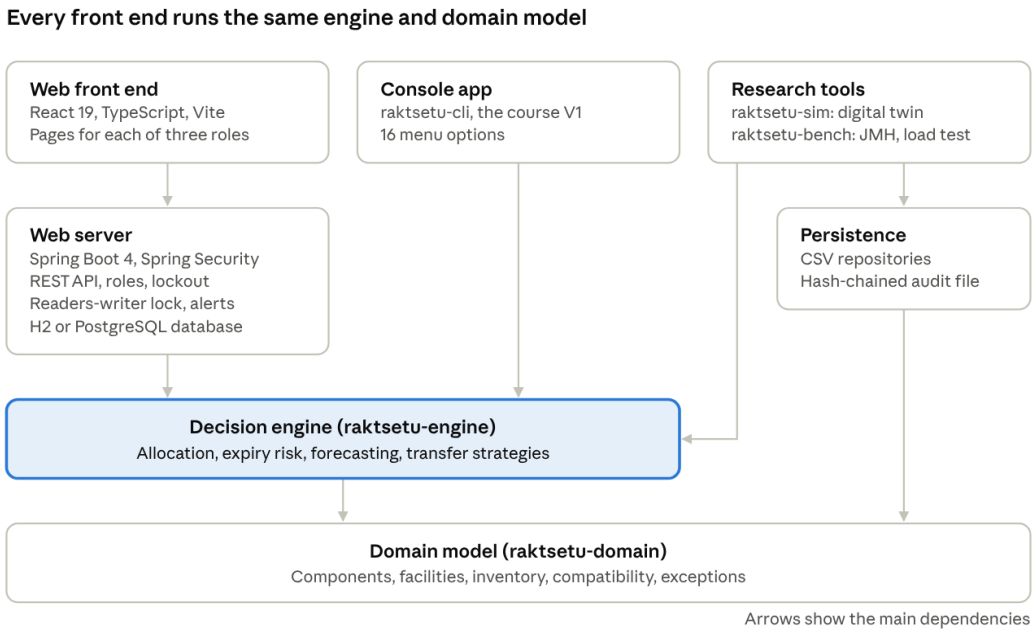
2.3 Non-functional requirements

Quality	Requirement	How it is met
Correctness	Business rules verified automatically	189 tests; 80% line-coverage gate per module

Quality	Requirement	How it is met
Security	Session sign-in, CSRF protection, role checks, lockout	Spring Security; 429 after 5 failed sign-ins in 15 minutes
Integrity	Tampering with history is detectable	SHA-256 hash chain; health check reports DOWN on tampering
Performance	Interactive at state scale	Emergency allocation 0.13 ms at 1,000 centres
Usability	One command to run; no demo data	Single JAR with the web front end; first-run setup page
Portability	Runs on Windows, macOS, Linux	Java 21+; embedded H2 database or PostgreSQL

3. System design

RaktSetu is a modular monolith: seven Maven modules and a React front end share one domain model and one decision engine, so the console app, the web app and the simulator all make the same decisions.



module architecture · 8 modules, arrows point to what each uses

The domain model has no dependencies; the engine depends only on it; everything else sits on top.

3.1 How a change is saved

The server keeps the network in memory for speed and treats the database as the source of truth. Every change runs through one method: the engine applies it, the audit events it produced are hash-chained and written, and the database tables are updated, all in one transaction. If saving fails, the in-memory network is reloaded from the database, so the two never drift apart.

3.2 Database

Flyway migrations V1 to V5 create the tables: facilities, routes, units, usage, wastage, the audit log, requests and their units, transfers, notifications, users and the alert outbox. The embedded H2 database needs no installation; PostgreSQL is used in production and in CI.

3.3 Security

- Session sign-in with BCrypt-hashed passwords; no default or demo accounts.
- CSRF protection on every change (token in the `XSRF-TOKEN` cookie, echoed in a header).
- Role checks on every endpoint; centre and hospital staff act only for their own facility.
- One-time passwords for new and reset accounts, with a forced change at first sign-in.
- Lockout after 5 failed sign-ins per account or 20 per address in 15 minutes.
- Phone numbers are masked in logs and screens; provider keys come only from environment variables.

4. Object-oriented design

Every OOP concept in the course appears in working code, and each was chosen for a reason in the blood-bank domain, not as an exercise.

4.1 The four pillars

Concept	Where in the code	Why it fits
Abstraction	`BloodComponent` and `Facility` are abstract classes	Code works with "a component" or "a facility" without knowing which kind
Encapsulation	`Inventory` keeps each component and group in a private, always-sorted `TreeSet`; `SlidingWindowAttemptPolicy` hides its failure queues	Stock can only change through methods that keep it in expiry order
Inheritance	`WholeBlood`, `RedCellConcentrate`, `Platelets`, `FreshFrozenPlasma` extend `BloodComponent`; `HubBloodCentre`, `StorageCentre`, `Hospital` extend `Facility`; all business exceptions extend `RaktSetuException`	Shared state (ID, group, dates) is written once
Polymorphism	`getShelfLifeDays()`, `getStorageCondition()`, `canStore()`, `canCollect()`	Expiry is computed the same way for every component: 5, 35, 42 or 365 days come from the subclass

4.2 Interfaces

Interface	Implementations	Purpose
AuditTrail	AuditLogger (hash-chained file), InMemoryAuditTrail, CapturingAuditTrail (database), NoOpAuditTrail (benchmarks)	The engine records events without knowing where they are stored
Repository<T extends Identifiable>	CsvRepository<T>	One generic class stores any identifiable entity
RedistributionStrategy	NearestFirstStrategy, ExpiryFirstStrategy	The administrator picks a planning policy at run time
DemandForecaster	Moving average, exponential smoothing, Croston/SBA, day-of-week, seasonal smoothing	The risk check and planner ask "a forecaster" for demand
AlertChannel	SmsChannel, WhatsAppChannel, LogChannel	Adding an SMS provider is one new class
LoginAttemptPolicy	SlidingWindowAttemptPolicy	The lockout rule can be replaced without touching sign-in
Identifiable, ExpiryTracked, Transfusable, Exportable, DistanceProvider, StockDataSource	Domain classes	Small single-purpose contracts (interface segregation)

4.3 Design patterns

- **Strategy:** redistribution strategies, demand forecasters, alert channels, login-attempt policies.
- **Template Method:** AbstractRedistributionStrategy.plan() is final and fixes the algorithm; subclasses supply only order() and choose(). HttpAlertChannel.send() shares timeouts and error handling while SMS and WhatsApp build their own requests.
- **Factory:** each DemandForecaster creates Forecast objects (the records Flat and Weekly); ComponentType.newUnit() and FacilityType.newFacility() create the right subclass.
- **Repository:** Repository<T> for CSV files and the repositories in the server's db package.
- **Transactional outbox:** an alert is saved in the same database transaction as its cause and sent later, with retries.
- **Dependency injection:** collaborators arrive through constructors, so tests pass a fixed Clock and fake providers.

4.4 Exceptions

A checked hierarchy carries business rules: RaktSetuException with subclasses InsufficientStockException, IncompatibleBloodException, UnitExpiredException, FacilityNotFoundException and InvalidDataException. The web layer maps each to a clear JSON error and HTTP status. Two exceptions carry state that drives behaviour: TooManyAttemptsException.retryAfter becomes the Retry-After header, and AlertDeliveryException.retryable decides whether an alert is retried.

4.5 Other Java features

- Generics with bounds (Repository<T extends Identifiable>), records for immutable values, enums with behaviour (BloodGroup, ComponentType).
- Collections chosen for their cost: EnumMap, TreeSet (stock in expiry order), ArrayDeque (sliding window), ConcurrentHashMap (thread-safe lockout).
- Concurrency: a readers-writer lock around the shared network, volatile fields, scheduled jobs, and virtual threads in the load test.
- Streams and lambdas, file I/O for CSV and the audit file, and SHA-256 hashing.

5. UML design documents

Eight PlantUML diagrams describe the system; they live in docs/uml/, are reviewed like code, and are updated in the same change as the code they describe

Diagram	File	What it shows
Use cases	use-case.puml	The three actors and what each can do, plus the simulator
Packages	packages.puml	Module dependencies: domain at the bottom, server and console on top
Domain classes	domain-classes.puml	Components, facilities, inventory, requests, transfers, interfaces and exceptions
Engine classes	engine-classes.puml	Allocation, expiry risk, freshness, the strategies (Template Method), transfers
Emergency request	sequence-emergency-request.puml	Sequence of an emergency request from the hospital to issued units
Security and monitoring	server-security-monitoring.puml	Sign-in lockout (Strategy), health check, metrics, the readers-writer lock, incremental audit check
Urgent alerts	alerts.puml	Alert channels (Strategy and Template Method), the transactional outbox and the dispatcher

Diagram	File	What it shows
Demand forecasting	forecasting.puml	The forecasters, the Forecast objects they create, and where the engine uses them

To render them, open the files with the PlantUML plugin for IntelliJ or VS Code, or run java -jar plantuml.jar -tpng docs/uml/*.puml.

6. Implementation

Six algorithms carry the system; each is short, deterministic and covered by tests.

6.1 Allocating a request (RequestService)

1. Remove expired units (cheap: each inventory remembers its earliest expiry and skips the scan when nothing can have expired).
2. Find centres with a route to the hospital within 90 minutes, from a neighbour index built once from the route table. Unverified centres go last.
3. Order donor groups: identical group first, then compatible ones; O- last for patients who are not O-.
4. Emergency: nearest centre first. Routine: exact group first across centres.
5. Take soonest-expiring units first. If the whole quantity cannot be met, issue nothing and record the request as unmet.

6.2 Expiry risk (ExpiryRiskAnalyzer)

Sort a centre's usable units of one component and group by expiry. The k-th unit is at risk if the demand forecast over its remaining days is less than k:

$$k > D(\text{daysLeft}_k), \qquad D_{\text{moving average}}(n) = \bar{u} \cdot n$$

Only units expiring within 7 days are reported, so long-lived plasma is not flagged months ahead.

6.3 Transfer planning (AbstractRedistributionStrategy)

For each at-risk unit, in the order the strategy chooses, a destination is feasible if it stores blood, its data is verified, the unit arrives with at least a day left, and it has spare demand:

$$\text{spare} = \lfloor D(\text{days on arrival}) \rfloor - \text{usable stock} - \text{already sent}$$

Nearest-First sends each unit to the closest feasible centre; Expiry-First places the soonest-expiring units first. Transfers are same-group only and need staff approval.

6.4 Tamper-evident audit log (`AuditHashing`)

Each entry stores the hash of its own fields and the previous entry's hash, so editing, deleting or reordering any entry breaks every hash after it:

$$h_n = \mathrm{SHA256}(n \,\Vert\, \text{event}_n \,\Vert\, h_{n-1}), \qquad h_0 = \text{GENESIS}$$

The overview checks only entries added since its last check; the whole chain is re-checked every five minutes, on the audit page and by the health check.

6.5 Urgent alerts (transactional outbox)

When an urgent notification is saved, an outbox row per recipient and channel is written in the same transaction. A background dispatcher sends due rows every few seconds, outside the network lock. Failures that may pass (HTTP 5xx, 429, network errors) are retried after 30 s, 1, 2 and 4 minutes, up to 5 tries; a wrong number or rejected key fails at once.

6.6 Concurrency (`NetworkLock`)

The in-memory network is shared by every user. A fair readers-writer lock lets any number of reads run together while a change has the network to itself. The daily roll-over runs before a read starts, because Java's lock cannot be upgraded from read to write.

7. Testing and quality

195 automated tests run on every change (189 in Java, 6 in the web front end), and the build fails if any module drops below 80% line coverage.

Module	Tests	What they check
raktsetu-server	62	API, sign-in and roles, CSRF, workflows, database write-through, lockout, health and metrics, concurrency, alerts end to end

Module	Tests	What they check
raktsetu-engine	43	Allocation, compatibility order, expiry risk, strategies, transfers, forecasters (hand-checked values)
raktsetu-domain	33	Shelf lives, compatibility rules, inventory order and expiry, parsing
raktsetu-sim	22	Simulator, digital twin, demand patterns, the forecasting study end to end
raktsetu-persistence	15	CSV parsing and round trips, corrupted rows reported, audit file detects edited and deleted entries
raktsetu-cli	7	Every menu option, saving and reloading, bad input never crashes, tampered audit log detected
raktsetu-bench	7	Synthetic network generator, latency percentiles
web (Vitest)	6	Formatting helpers

How quality is enforced

- **Coverage gate:** JaCoCo fails the build below 80% line coverage per module.
- **Continuous integration:** GitHub Actions builds and tests on Java 21 and 25, and runs the server tests a second time against PostgreSQL 17.

- **Equivalence checks:** each performance change and the forecasting refactor were checked by re-running the 30-day simulation for 5 seeds and 3 policies; the output was byte-identical before and after.
- **Concurrency test:** 16 threads raise, cancel and read at once; stock must end exactly as it started and the audit chain must be intact.
- **Fake providers:** SMS and WhatsApp requests are checked byte for byte against a local fake server, including retries and failures.
- **Browser checks:** the full first-run flow (setup, facilities, people, one-time password, a request from hospital to dispatch and receipt, an alert) was exercised in a real browser on an empty database.

8. Performance

Profiling found the bottlenecks, and fixing them made the engine 5 to 66 times faster at state scale and the server 4.4 times faster under load, with identical results.

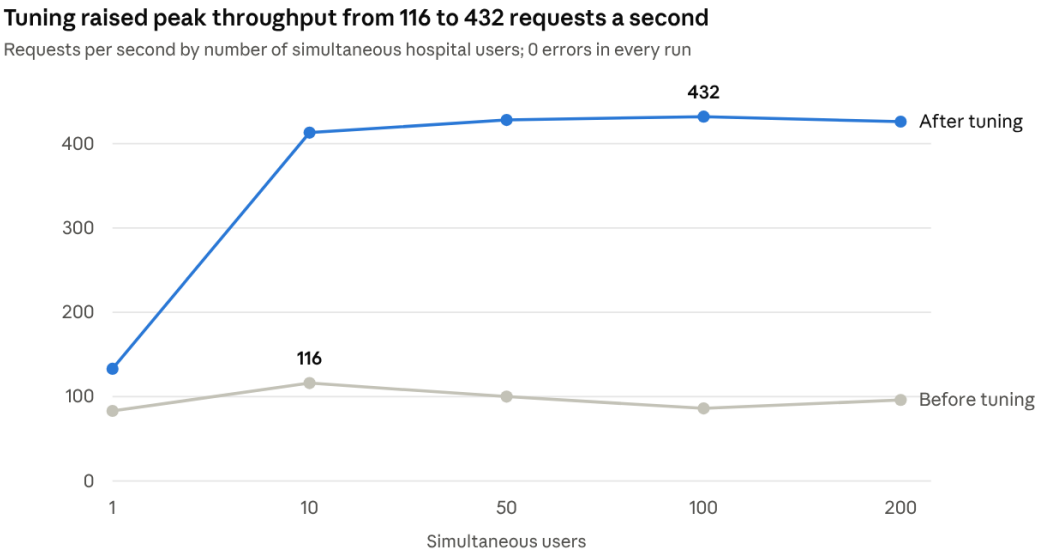
8.1 Engine (JMH benchmarks)

Generated networks with 300 units per centre and two hospitals each; 1,000 centres means 3,000 facilities and 300,000 units.

Operation at 1,000 centres	Before	After	Speed-up	Main fix
Serve an emergency request	8.50 ms	0.13 ms	66x	Skip the expiry scan when nothing can have expired; search only nearby centres
Find units at risk of expiring	421 ms	76 ms	5.5x	Expiry date computed once; stock kept in a `TreeSet` instead of a `PriorityQueue`

Operation at 1,000 centres	Before	After	Speed-up	Main fix
Plan transfers for the network	31.6 s	0.67 s	47x	Neighbour index; per-plan lookup tables (planning time now grows linearly, not with the square of the network)

8.2 Server (HTTP load test)



docs/benchmarks/load-before.md and load-after.md · 20 centres, 40 hospitals, 30 s per level

The load test signs in hospital staff through the real API and runs a realistic mix: 85% reads, 15% raising and cancelling an emergency request. Three fixes produced the gain: the overview stopped re-hashing the whole audit log on every page (incremental verification), a readers-writer lock let reads run in parallel, and a list page went from 51 database queries to 2. At 200 users the slowest 1% of

responses fell from 3.16 s to 0.74 s. Throughput now levels off because each change holds the write lock while it saves to the database.

9. Research results

In a digital twin of a Pune district network, redistribution cut expired units by about 60%, while the choice of demand forecaster changed them by under 20 units in 120 days.

9.1 Set-up

The digital twin simulates 10 blood centres and 22 hospitals (synthetic facilities at real towns, OSRM road times) day by day. It keeps the true stock separate from what the planner sees through centres' reports. Each policy replays the same random days (common random numbers), so differences between policies are paired; 20 replications give 95% confidence intervals. Three demand scenarios were tested: flat, a weekday pattern, and a weekday pattern with a dengue-season platelet surge. Their magnitudes are illustrative.

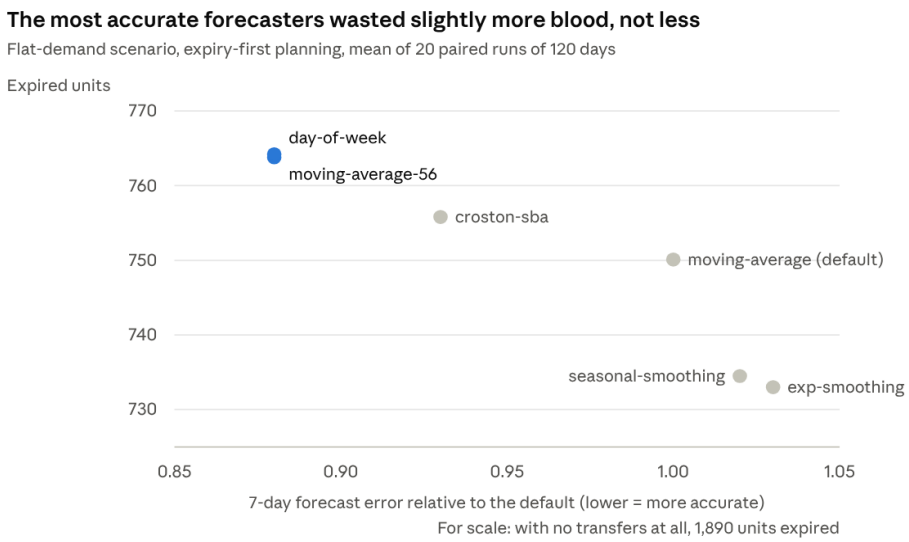
9.2 Redistribution

Scenario	Expired units: no transfers to expiry-first	Unmet demand (share of units requested)
Flat demand	1,890 to 750 (-60%)	2.13% to 1.02%
Weekday pattern	1,913 to 757 (-60%)	2.09% to 1.02%
Weekday + dengue surge	1,509 to 342 (-77%)	9.81% to 8.53%

When supply is adequate, moving at-risk units to where they will be used cuts waste by about 60% and halves shortages. In the dengue scenario platelet demand doubles while supply does not, so most of the shortage remains.

9.3 Forecasting

Six forecasters were compared, including a 56-day moving average as a control for window length.



docs/research/forecasting/results.md · Pune pilot network, flat demand, 20 replications

- A longer history made forecasts up to 13% more accurate; modelling the weekday added about 1 point beyond that, and Croston/SBA did not help rare groups.
- More accurate forecasts did not give better plans: wastage tracked how many units were moved (correlation -0.93 to -0.96), not forecast error. Forecasts for this decision should therefore be judged by the wastage and shortages they lead to.

Full method, tables and threats to validity are in `docs/research/forecasting/`.

10. User guide

The only requirement is Java 21 or newer; one command builds everything, and one starts the web app at http://localhost:8080.

10.1 Build and run

```
./mvnw package -DskipTests
java -jar raktsetu-server/target/raktsetu-server.jar
```

On Windows, double-click `run-web.bat` instead. Maven downloads its own Node.js to build the web front end, so nothing else needs installing. Options: `--raktsetu.data-dir=<folder>` chooses where the database lives; `--server.port=<port>` changes the port.

The console application (the course V1 deliverable) runs with `java -jar raktsetu-cli/target/raktsetu.jar` and offers 16 menu options on a sample network.

10.2 First use

1. **Setup:** the first visit asks for the first network administrator's name, username and password (at least 10 characters, with a letter and a digit).
2. **Facilities:** add the hub, storage centres and hospitals by clicking each location on the map. Road routes are worked out automatically and can be corrected by hand.
3. **People:** create an account for each member of staff. RaktSetu shows a one-time password once; the person chooses their own at first sign-in.
4. **Stock:** centre staff log collections (with the donation number on the bag, or numbered automatically) and report stock daily.
5. **Requests:** hospital staff raise emergency or routine requests; the reserved centres are notified, dispatch, and the hospital confirms receipt.
6. **Transfers:** the administrator generates a plan, then approves or rejects each proposed transfer.

10.3 Optional settings

- **Urgent alerts:** each person can add a mobile number on their Account page. To send real SMS or WhatsApp messages, set the Twilio or Meta Cloud API keys as environment variables (listed in the README).
- **PostgreSQL:** `docker compose up --build`, or start the JAR with `--spring.profiles.active=postgres`.
- **API documentation:** http://localhost:8080/api-docs; health check: http://localhost:8080/actuator/health.

11. Limitations, future work and conclusion

RaktSetu works end to end and its design choices are measured, but its outcome figures come from simulation and still need real data.

11.1 Limitations

- **Synthetic data:** the Pune network places synthetic facilities at real towns; demand, supply and the weekday and dengue patterns are illustrative, not calibrated.
- **Censored usage:** usage is recorded as units issued, so demand that went unmet during a stock-out is invisible to forecasting.

- **One server:** the network lives in one server's memory behind one lock; throughput levels off at about 430 requests per second on a laptop because each change holds the write lock while it saves.
- **Decision support only:** every transfer needs human approval, and compatibility rules must be confirmed against current NACO/NBTC guidelines before any clinical use.
- **Alerts need an account:** real SMS or WhatsApp messages need a Twilio or Meta account; without one, alerts go to the server log.

11.2 Future work

1. Calibrate demand, supply and wastage against real usage data, for example from e-Raktkosh or a partner hospital.
2. Repeat the digital-twin study at 50-70% reporting compliance, and forecast from requests rather than issues to remove censoring.
3. Save changes outside the write lock and partition the network by region to scale beyond one server.
4. Read ISBT 128 bag barcodes and exchange data in e-Raktkosh's format.
5. Host a public demonstration instance.

11.3 Conclusion

RaktSetu shows that an object-oriented design can carry a real decision problem from domain model to working product. On the simulated network, moving at-risk units to where they will be used cut expired units by about 60% and halved shortages. The same model runs in a console application, a multi-user web app and a research simulator, all held to one test suite.

The forecasting study adds a finding worth keeping: a more accurate demand forecast did not produce a better plan, so forecasts for this decision should be judged by the wastage and shortages that result, not by forecast error.

References

1. J. D. Croston, "Forecasting and stock control for intermittent demands," *Operational Research Quarterly*, vol. 23, no. 3, pp. 289-303, 1972.
2. A. A. Syntetos and J. E. Boylan, "The accuracy of intermittent demand estimates," *International Journal of Forecasting*, vol. 21, no. 2, pp. 303-314, 2005.
3. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
4. Ministry of Health and Family Welfare, Government of India, e-Raktkosh: Centralized Blood Center Management System. [Online]. Available: https://eraktkosh.mohfw.gov.in/ (accessed 1 October 2026).
5. Project OSRM, Open Source Routing Machine, used for road distances and travel times. [Online]. Available: https://project-osrm.org/ (accessed 1 October 2026).
6. OpenJDK, Java Microbenchmark Harness (JMH) 1.37. [Online]. Available: https://github.com/openjdk/jmh (accessed 1 October 2026).
7. Spring Boot 4.1 and Spring Security 7 reference documentation. [Online]. Available: https://docs.spring.io/spring-boot/ and https://docs.spring.io/spring-security/reference/ (accessed 1 October 2026).
8. Rajya Sabha, Unstarred Question No. 1882, answered 4 August 2026, citing e-Raktkosh data on blood collection and discards. Reported in Factly, "Blood collection rose 24% in India, while discards more than doubled." [Online]. Available: https://factly.in/blood-collection-rose-24-in-india-while-discards-more-than-doubled/ (accessed 1 October 2026).